Project 7.1 — LinkKeeper-Personal-Bookmarks-MCP-Server

A practice twin of Project 7 — MCPForge. Same skills — build a real **MCP server** other people's agents can connect to: the three MCP building blocks (**Tools**, a **Resource**, a **Prompt**), a real **SQLite** store that survives a restart, a clean error returned as a readable string, and a **client** that discovers and uses everything over the protocol. A **new, real-world job**. **There are no steps here on purpose.** Read the story, build it yourself, and only open the hints or the solution if you get stuck.

Type: MCP server (Tools + Resource + Prompt) + a client that uses it · **Stack:** Python, `mcp` (the official MCP Python SDK), SQLite · **Level:** Intermediate–Advanced

> **Companion to Project 8.1 — OpsPilot** (an MCP *client*): this project builds the *server* side.
> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

You keep a pile of useful links — docs, articles, tools — scattered across browser tabs and chat messages. You want an assistant to *save* them, *find* them later, and *turn them into a reading list* — but you don't want to hard-wire that capability into one assistant. You want it to live as a **service** any MCP-speaking agent (Claude Desktop, an IDE, your own client) can plug into.

You are going to build **LinkKeeper**: a bookmarks **MCP server**. It exposes its capability over the Model Context Protocol in the three standard shapes, so any client discovers and uses it without knowing a thing about your code:

- **Tools** — `add_bookmark(url, title, tag)` and `search_bookmarks(query)`: actions a client's agent can call.
- **A Resource** — `bookmarks://all`: readable content (every saved bookmark) a client can fetch by its URI.
- **A Prompt** — `reading_list`: a reusable, named prompt template the server offers, so a client can ask "turn my bookmarks into a reading list" without writing the wording itself.

Behind the tools is a real **SQLite** store, so bookmarks saved in one session are still there the next time the server starts. And a bad call — saving a bookmark with an empty URL — comes back as a clear `"Error: ..."` string the client can read, not a crash.

The one lesson of this project: an MCP server is how you expose a capability — not as a library someone imports, but as **Tools, a Resource, and a Prompt** any client can discover at runtime and

use over one standard protocol. Project 8.1 built the client that consumes servers like this; here you build the server.

How it works, from a client call to a stored bookmark (picture the whole machine)

Read this once and the whole project should form in your head.

The server is a single `MCPServer` instance. You register its capability with three decorators: `@mcp.tool()` on each action function, `@mcp.resource("bookmarks://all")` on the function that returns all bookmarks as text, and `@mcp.prompt()` on the function that returns the reading-list prompt wording. Then `mcp.run(transport="stdio")` starts it listening on stdin/stdout. **One rule that bites everyone:** the server logs to **stderr**, never `print()` to stdout — stdout *is* the protocol channel, and a stray print there corrupts it.

A client connects over stdio and calls `initialize()` (the handshake). Now it can **discover**: `list_tools()`, `list_resources()`, `list_prompts()` — it sees what's on offer without importing anything from the server. To **use** a tool it calls `call_tool("add_bookmark", {...})`; the server runs the function against the **SQLite** store and returns a result whose `.content[0].text` is the confirmation. To use the resource it calls `read_resource("bookmarks://all")` and reads `.contents[0].text`. To use the prompt it calls `get_prompt("reading_list")` and reads the message text.

The tools go through a tiny `db` layer: `add_bookmark` inserts a row (raising on an empty URL, which the tool catches and returns as `"Error: url cannot be empty"`), `search_bookmarks` scans rows for a keyword, and the resource returns them all. Because it's SQLite on disk, stopping and restarting the server keeps every bookmark.

That's the whole machine: **register Tools + a Resource + a Prompt on one `MCPServer`, back the tools with SQLite, log to stderr, run over stdio — and a client connects, discovers all three, and uses them, with a bad call coming back as a readable error.**

What you are building (the pieces)

- An `MCPServer` exposing two **tools**, one **resource** (`bookmarks://all`), and one **prompt** (`reading_list`), run over stdio.
- A **SQLite store** (`db.py`) behind the tools, so bookmarks persist across restarts.
- A **clean error path**: an empty URL is caught and returned as a `"Error: ..."` string (not a raised exception the client sees as a generic failure).
- A **client** (`client.py`) that connects, discovers tools/resources/prompts, and uses all three.
- The Doc01 basics: config, a named exception, and a logger that writes to **stderr**.

Inputs: tool calls and resource/prompt requests from any MCP client. **Outputs:** confirmations and content as text; an error as a readable string. **Rules:** three building blocks exposed; the store survives a restart; stdout stays clean (log to stderr); a bad call never crashes the server.

A real example to build and test against

- Client calls `add_bookmark("https://docs.python.org", "Py Docs", "ref")` → `"Saved: Py Docs [ref]"`, stored in SQLite.
- Client calls `add_bookmark(" ", "oops")` → `"Error: url cannot be empty"` (the text starts with `Error`).
- Client calls `search_bookmarks("py")` → the matching bookmark.
- Client reads the `bookmarks://all` resource → every saved bookmark as text.
- Client gets the `reading_list` prompt → the reusable prompt wording.
- Restart the server, search again → the bookmark is **still there** (SQLite persisted it).

How to practice this

1. Read this story. **Don't open the solution yet.** (Read Doc06's MCP sections if the protocol is new.)
2. Plan it on paper: the three building blocks, the SQLite schema, where the empty-URL check lives, and what the client calls to discover vs. use.
3. Build it yourself. A good order: a minimal server with one tool → the SQLite store + search → the resource + the prompt → the client that uses all three. Your call.
4. Test it on the real example: add, the empty-URL error, search, read the resource, get the prompt, and the restart-persistence check.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

MCP is how production assistants increasingly get their capabilities: a team exposes a service as an MCP server, and any agent — Claude Desktop, an IDE, a custom client — plugs in without a bespoke integration. Building the *server* side proves you understand the contract from the provider's end: Tools for actions, a Resource for readable content, a Prompt for reusable wording, discovered at runtime over one protocol. The habits here — a real store that persists, errors returned as readable strings, and keeping stdout clean for the protocol — are exactly what separates a toy server from one another team would actually connect to. Build LinkKeeper and you can expose any capability as an MCP server.

Done when

- The server exposes two tools, the `bookmarks://all` resource, and the `reading_list` prompt, and a client discovers all three.
- Bookmarks are stored in SQLite and survive a server restart.
- An empty-URL call comes back as a readable `"Error: ..."` string, not a crash.
- The server logs to stderr (stdout stays clean for the protocol).
- Your tests pass: settings, the db layer (add/search/all + the empty-URL error, no network), and a live client run against the real server.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.